# PROJECT REPORT

AI-Based Phishing Website Screenshot Detection Using Computer Vision

**Computer Vision – VITyarthi Flipped Course Evaluation**

**Submitted By**

Vipul Kumar Verma

**( 24BAI10619 )**

18th September 2026

## Abstract

Phishing websites imitate legitimate websites to deceive users. This project develops a computer-vision-based system that classifies a website screenshot as legitimate or phishing. It uses image preprocessing, Histogram of Oriented Gradients (HOG) feature extraction, and a Support Vector Machine (SVM) classifier. A command-line interface accepts a screenshot and reports the predicted class with a confidence value. The implementation uses a Kaggle screenshot dataset containing 1,697 images and evaluates the model using accuracy, precision, recall, F1-score, and a confusion matrix.

## 1. Project Description and Outline

### 1.1 Introduction

Phishing is a cybersecurity problem in which deceptive websites are designed to resemble trusted websites. Since webpages have recognizable visual structures, screenshots can provide useful information for a computer-vision-based classification system. This project analyzes only the visual appearance of a webpage screenshot.

### 1.2 Problem Statement

Develop a computer-vision-based system that identifies whether a website screenshot belongs to a legitimate or phishing website by processing visual information and returning a binary prediction with confidence.

### 1.3 Objectives

• Apply computer vision to phishing detection.

• Preprocess website screenshots consistently.

• Extract visual features using HOG.

• Train an SVM classifier for binary classification.

• Evaluate accuracy, precision, recall, and F1-score.

• Provide a simple command-line prediction interface.

• Demonstrate automated testing.

### 1.4 Scope and Target Users

The project focuses on screenshot-based visual classification and does not inspect URLs, HTML, JavaScript, domain information, certificates, or live website behavior. Target users include students, learners, and developers experimenting with visual phishing detection.

### 1.5 Project Organization
The repository is organized into modules for dataset loading, preprocessing, feature extraction, model training, prediction, evaluation, utilities, and testing. The root main.py file provides the command-line entry point.

# 2. Related Work Investigation
Automated phishing detection can use URL, domain, HTML, JavaScript, network, behavioral, or visual information. This implementation intentionally uses webpage screenshots so that Computer Vision concepts remain central. HOG provides local edge and gradient features, while SVM provides a classical binary classification baseline.

# 3. Requirements and Requirement Artifacts
## 3.1 Functional Requirements

| ID | Requirement |
|---|---|
| FR1 | Load legitimate and phishing screenshots. |
| FR2 | Resize, grayscale and normalize images. |

| ID | Requirement |
|---|---|
| FR3 | Extract HOG features. |
| FR4 | Train an SVM using a stratified 80:20 split. |
| FR5 | Evaluate accuracy, precision, recall, F1-score and confusion matrix. |
| FR6 | Predict a new screenshot through the CLI. |
| FR7 | Run automated Pytest testing. |

## 3.2 Non-Functional Requirements

| Category | Requirement |
|---|---|
| Usability | Simple command-line operation. |
| Performance | CPU-friendly processing. |
| Maintainability | Modular source files. |
| Reliability | Validation and error handling. |
| Portability | Python-based execution. |
| Testability | Automated Pytest test. |

## 3.3 Technical Requirements
• Python 3.12 environment.

• Standard desktop/laptop; no GPU is required for the implemented HOG + SVM pipeline.

• OpenCV, NumPy, scikit-image, scikit-learn, Joblib, Pytest, Matplotlib and Seaborn.

# 4. Design Methodology and Novelty
## 4.1 Methodology
Dataset preparation → image validation → resize to 256 × 160 → grayscale and normalization → HOG feature extraction → stratified 80:20 split → RBF SVM with balanced class weights → evaluation → saved model → new-image prediction.

## 4.2 System Architecture

**Input Screenshot → Image Preprocessing → HOG Feature Extraction → SVM Classifier → Legitimate / Phishing → Confidence Score**

## 4.3 Component Structure

| Component | Responsibility |
|---|---|
| data_loader.py | Load images and labels. |
| preprocess.py | Preprocess images and extract HOG. |
| train_model.py | Train and save SVM. |
| predict.py | Predict new screenshots. |
| evaluate.py | Calculate metrics. |
| utils.py | Supporting utilities. |
| tests/test_preprocess.py | Test feature extraction. |

| Component | Responsibility |
|---|---|
| main.py | CLI entry point. |

## 4.4 Design Artifacts

The project includes a problem statement, objectives, functional and non-functional requirements, system architecture, process workflow, component structure, dataset description, model selection rationale, and evaluation methodology. A usecase view and sequence view are also documented in this report.

# 5. Technical Implementation and Analysis

## 5.1 Dataset

The local implementation uses 1,147 legitimate screenshots and 550 phishing screenshots, for 1,697 images total. The stratified 80:20 split produced 1,357 training samples and 340 test samples.

| Class | Images |
|---|---|
| Legitimate | 1,147 |
| Phishing | 550 |
| Total | 1,697 |

## 5.2 Image Preprocessing and HOG

Images are loaded with OpenCV, resized to 256 × 160 pixels, converted to grayscale, and normalized. HOG uses 9 orientations, pixels_per_cell=(16,16), cells_per_block=(2,2), and L2-Hys normalization to represent local edge and gradient structures.

## 5.3 Model Selection and Rationale

An RBF-kernel SVM was selected for binary classification using the extracted feature vectors. Balanced class weights compensate for unequal class sizes. Probability estimation is enabled so the CLI can display a confidence value.

## 5.4 Experimental Results

| Metric | Result |
|---|---|
| Accuracy | 67.65% |

| | |
|---|---|
| Precision | 0.5000 |
| Recall | 0.6727 |
| F1 Score | 0.5736 |

### 5.5 Confusion Matrix

| | **Predicted Legitimate** | **Predicted Phishing** |
|---|---|---|
| Actual Legitimate | 156 | 74 |
| Actual Phishing | 36 | 74 |

The model correctly classified 156 legitimate and 74 phishing screenshots. It misclassified 74 legitimate screenshots as phishing and 36 phishing screenshots as legitimate.

### 5.6 Example Prediction

A Google homepage screenshot was supplied to the trained application. Recorded output:

==========

AI-BASED PHISHING SCREENSHOT DETECTOR

==========

Input : Screenshot 2026-09-13 103407.png

Prediction : LEGITIMATE

Confidence : 76.96%

==========

# 6. Outcome and Applicability

• Complete screenshot-based phishing classification pipeline.

• HOG visual feature extraction and SVM classification.

• Evaluation on 340 held-out screenshots.

• Command-line prediction interface.

• Automated preprocessing testing.

• Modular public GitHub repository.

### 6.1 Challenges and Limitations

• Dataset organization and unequal class sizes.

• Python module and import-path consistency during testing.

• Screenshot-only analysis without URL or live webpage information.

• 67.65% accuracy means meaningful misclassification remains.

• Confidence should not be treated as a security guarantee.

# 7. Testing and Validation

### 7.1 Unit Testing

An automated Pytest test verifies that the preprocessing and HOG feature-extraction pipeline can process a valid image and produce a non-empty feature vector. Test result: **1 passed in 0.59s**.

### 7.2 Functional Validation

| **Test Area** | **Expected Result** | **Observed Result** |
|---|---|---|

| | | |
|---|---|---|
| Dataset loading | Images and labels discovered. | Successful |
| Feature extraction | Valid feature vector produced. | Successful |
| Model training | SVM trained and saved. | Successful |
| Evaluation | Metrics and confusion matrix produced. | Successful |
| Prediction | Class and confidence returned. | Successful |
| Automated test | Pytest passes. | 1 passed in 0.59s |

### 7.3 Error Handling and Validation

The preprocessing pipeline checks whether an image can be loaded before extracting features. Invalid or unreadable inputs are rejected rather than silently converted into invalid feature vectors. The prediction flow expects a valid image path and trained model.

### 7.4 Version Control

Git was used to initialize, track, commit, branch, and push the project. Large dataset/model artifacts are excluded through .gitignore while code and documentation remain in the public repository.

## 8. Conclusion and Recommendations

The project demonstrates a complete Computer Vision workflow for phishing screenshot classification, covering dataset preparation, preprocessing, HOG feature extraction, SVM classification, evaluation, prediction, automated testing, and documentation. The model achieved 67.65% accuracy on the held-out test set, establishing a working baseline while showing that further improvement is needed for practical security deployment.

### 8.1 Future Enhancements

• Compare with CNN/deep-learning models.

• Use larger and more diverse datasets.

• Improve class balance and augmentation.

• Tune feature and classifier parameters.

• Combine screenshot features with URL, HTML and domain features.

• Add explainable AI techniques.

• Develop a web or graphical interface.

### 8.2 Learning Outcomes

• Image preprocessing and HOG feature extraction.

• SVM binary classification and model evaluation.

• Python modular programming.  • Automated testing with Pytest.

• Git and GitHub version control and documentation.

### 8.3 GitHub Repository

**https://github.com/vipul-09/phishing-website-screenshot-detection-**

## 9. References


• Kaggle — “Phishing sites screenshot” dataset.

• Scikit-learn documentation — Support Vector Machines and evaluation.

• OpenCV documentation — image processing.

• Scikit-image documentation — Histogram of Oriented Gradients.

• Pytest documentation — automated testing.

**Declaration:** This report describes the implemented project, its measured results, and its current limitations. The reported metrics correspond to the implemented 80:20 stratified evaluation procedure.